Technical Assessment Pack

Senior Rails Developer

Estimated duration to complete: 4 hours, 3 business days to submit

Contents

Stage one, take-home

- Part 1, candidate brief *(share)*
- Part 2, what the starter repo contains
- Part 3, scoring

Stage two, system design

- Part 1, the brief and three screens *(share)*
- Part 2, four curveballs
- Part 3, spares
- Part 4, reference model
- Part 5, scoring

Before you start

Two stages. Stage one is a four-hour take-home that tests whether they can build. Stage two is a forty-five minute conversation with no code that tests whether they can decide.

Different domains on purpose, so preparing hard for the first doesn't help with the second.

Only two things get shared with candidates: the brief in stage one, part 1, and the brief and screens in stage two, part 1. Everything else is internal, including both rubrics and the planted trap.

The starter repo doesn't exist yet. Part 2 says what it needs to contain, not how to write it. Budget half a day to stand it up, run both fixture generations end to end, and confirm the trap behaves. Stage two needs nothing but the screens and a whiteboard.

STAGE ONE, TAKE-HOME

Role: Senior Rails / Hotwire Developer

Time box: 4 hours

Stack: Rails 8, Hotwire, PostgreSQL, Sidekiq, Tailwind

PART 1, CANDIDATE BRIEF

Candidate-facing. Copy into the repo README.

Context

You're joining a team that maintains a film festival's public website. It's a Rails monolith that also acts as the editorial CMS, and it pulls programming data from an external festival-management system over an API.

That external system is the source of truth for films, venues and screenings. Our database holds a local copy so the site renders fast and stays up under load. Today that copy is refreshed by a nightly script a colleague wrote in a hurry, and it's been causing problems.

Your job is to replace it.

What we've given you

A working Rails 8 app with `Film`, `Venue` and `Screening` models, a mock external API at `/mock_api` that behaves like the real one including its failure modes, an existing `VenueSync` service and its tests, a screenings index with a filter form, and seed data.

Run `bin/setup`, then `bin/dev`.

What we'd like you to build

A programme sync. Pull screenings from `GET /mock_api/screenings` into our database. It's paginated, returns nested film and venue data, and behaves like a real third-party API: sometimes slow, sometimes failing partway through, and its records change between runs.

Running it twice must not create duplicates. Running it after upstream data changes must update the local copy. If the API fails partway, records already retrieved shouldn't be lost. It should run on a schedule as a background job, and we should be able to tell afterwards whether a run succeeded and what it did.

A filtered screenings list. The index at `/screenings` has a filter form that reloads the whole page. Make it update just the results. Filters are date, venue, and a text search across titles. Keep it server-rendered; we're a Hotwire shop and aren't looking for a client-side rendering layer.

A short README. Half a page. What you'd do differently with more time, anything in the existing code you'd change and why, and any assumptions you made.

Testing the mock API

Parameter	Effect

Parameter	Effect
`?page=2`	Pagination, 25 records per page
`?generation=2`	The dataset after upstream changes. Screenings have moved venue, some are cancelled, a film has been retitled, two screenings are new.
`?fail_after=8`	Returns 8 records, then a 500
`?slow=true`	Six-second delay

Sync `generation=1`, then `generation=2`, and check the database is right. Then try `generation=1&fail_after=8` and check nothing was lost.

What we care about

Correctness under failure, ahead of feature completeness. If you run short, a sync that handles the edge cases and a filter that doesn't quite work beats the reverse.

Tests for the sync are expected. Tests for the view layer aren't.

Submitting

A private git repo with your commits. Please don't squash.

PART 2, WHAT THE STARTER REPO CONTAINS

Internal. What the repo does, not how to write it.

Shape

Rails 8 on PostgreSQL with Tailwind, Sidekiq and RSpec. Postgres in Docker on a non-standard port so it won't collide with anything they already run. One bootstrap step from clone to booting app.

Three models, each with a unique index on its external identifier:

Model	Holds
`Film`	External ID, title, synopsis, runtime, year
`Venue`	External ID, name, address, capacity
`Screening`	External ID, film, venue, start time, status

No table for recording sync runs. They add that themselves.

The mock API

An endpoint inside the app that behaves like the real festival-management system: paginated, film and venue nested inside each screening, with query parameters that reproduce its failure modes on demand.

Parameter	Effect
`page`	Pagination, 25 per page
`generation`	Switches between the before and after datasets
`fail_after`	Returns N records, then a 500
`slow`	Six-second delay

Two fixtures sit behind it. Generation 1 is the starting state, around 60 screenings. Generation 2 is the same data after upstream changes:

Change	Catches
4 screenings move venue	Updates, not just inserts
3 screenings cancelled	Whether soft state changes propagate
1 film retitled, same ID	The `find_or_create_by(:name)` bug
1 venue renamed, same ID	Same, for venues
2 screenings added	That inserts still work
1 screening removed	Whether they handle deletions. Most won't, which is a discussion rather than a fail.

Get these exactly right. If the retitled film accidentally changes ID, the best check in the exercise stops working.

The planted trap

Ship an existing `VenueSync` service as inherited code. Don't mention it in the brief.

It matches venues on name rather than external ID, so a rename upstream creates a second record instead of updating the first. This is the bug class that produces manual ID reconciliation in real integrations. It also has no error handling, so one bad record abandons the batch.

Ship a passing test alongside it covering only the create path, so it looks maintained. The test passes and tests the wrong thing.

The screenings index

Server-rendered list, filter form for date, venue and text search. Three things are wrong with it and none are mentioned:

- A Turbo Frame wraps the results but the form doesn't target it, so filtering does a full reload
- The film and venue lookups in the loop are an N+1
- The search parameter is accepted by the form and ignored by the controller

The third is the quietest. Noticing it means actually using the feature rather than reading the code.

Observability

The brief asks them to report on sync runs, so treat it as expected work rather than a bonus. A timestamp and a status is the floor. Counts of created and updated records with captured errors is good. Structured enough to alert on is strong.

PART 3, SCORING

Score each area 1 to 4. 1 is absent or wrong, 2 attempted with gaps, 3 solid and would ship with review, 4 better than our current bar.

A. Idempotency

Look for an upsert keyed on the external identifier rather than a human-readable field.

A 4 handles the retitled film and renamed venue correctly and was tested by running the sync twice. A 2 has the right key but no test proving it. A 1 matches on title or name, or blindly creates.

Check after `generation=2`: one film with the new title, or two films?

B. Partial failure

A 4 isolates errors per record, captures and reports failures, commits what succeeded, and distinguishes a bad record from a dead API. A 3 uses per-page transactions so one page's failure doesn't lose earlier ones. A 2 wraps the whole sync in one transaction, which is safe but loses everything on any failure. For an hourly sync that means an hour of stale data. A 1 has no handling at all.

A and B carry the most weight.

C. Background job

A 4 is a Sidekiq job that's idempotent, retry-aware, and thinks about overlapping runs. A 2 has a job but no thought about concurrency. A 1 is a rake task with no retry story.

Bonus signal: do they say an hourly full sync is a poor fit for time-sensitive data and suggest event-driven updates. That's the real conversation on this project.

D. The trap

A 4 found it, explained the failure mode, mentioned it in the README, and didn't silently rewrite it. A 3 found and fixed it without explanation. A 2 copied the same pattern into their new sync. A 1 didn't notice.

The 2 matters most. Someone who inherits a bad pattern and propagates it is a real risk on a codebase they didn't write, which is this engagement.

E. Hotwire

A 4 targets the frame correctly, considered a Turbo Stream for filtering, maybe a Stimulus controller for debounced search. A 2 works but awkwardly. A 1 reached for `fetch` and manual DOM work, or added a client-side framework.

Watch for React habits. The 1 is a poor fit here but worth probing in the follow-up rather than auto-rejecting.

F. Query discipline

A 4 eager loads and mentions the N+1 they found in the existing code. A 2 added includes to their own code and missed the existing one. A 1 has N+1s everywhere.

G. Tests

A 4 covers failure paths, idempotency and changed data. A 2 is happy path only.

H. Communication

Is the README half a page and useful, or three pages of process narration? Can they say what they'd do differently without defensiveness or excessive self-criticism?

Decision guide

Outcome	Signal
Strong hire	4s in A and B, noticed the trap, README shows judgment about what to leave alone
Hire	3s across A, B and E. May have missed the trap, but nothing propagated.
Discuss	Solid engineering, weak Hotwire. Probe in stage two.
No	Propagated the trap, or no failure handling

Follow-up questions

Pick three for the start of stage two.

1. This sync runs hourly. It's festival week and a screening moves venue at 2pm. What happens, and what would you change?
2. What happens if two sync runs overlap?
3. You didn't handle records disappearing upstream. How would you decide between delete, soft-delete and leave?
4. What did you leave out, and how did you decide?
5. Anything in the existing code you'd have changed but didn't?

Five is the useful one. It surfaces what they noticed and their judgment about scope creep on someone else's code.

STAGE TWO, SYSTEM DESIGN

Format: 45 minutes, whiteboard or shared doc. No code, no IDE.

Focus: Relational modelling. Associations, cardinality, temporal data.

How to run it

Five minutes to set context and hand over the screens. Ten for them to sketch a first pass out loud. Then twenty-five for the four curveballs, and five at the end for their questions.

The domain is deliberately small. Three screens, one product, no side features. A candidate should be able to hold the whole thing in their head after one read, so that what you're testing is their modelling rather than their ability to absorb a spec under time pressure.

Don't correct a model that works. Several answers here are defensible and the signal is whether they can defend theirs.

Introduce curveballs one at a time and don't say how many are coming. Someone who designed for extensibility will absorb them cheaply. Someone who over-engineered upfront will have built for the wrong ones.

If they finish early, part 3 has spares. Don't plan to reach them.

PART 1, THE BRIEF

Candidate-facing. Hand over, or read aloud and share the screens.

City Arts Pass is a city-wide arts membership. Members pay an annual fee and get into events run by partner organisations: cinemas, theatres, galleries, music venues.

There's a web app and a mobile app. Here are three screens.

Screen 1, mobile, member home

Section	What it shows
Membership card	Plus · Anna Reid · Member since 2024
Included this month	Italian Film Weekend, Grand Cinema, Thu 8pm
	Late Opening, City Gallery, Fri 7pm
Saved	Five tiles: two events, a venue, an artist, a collection

Screen 2, web, event detail

Italian Film Weekend

Presented by City Arts Trust

Venue	Date and time	Availability
Grand Cinema	Thu 12 Mar, 8:00pm	12 left
Grand Cinema	Fri 13 Mar, 8:00pm	Sold out
Riverside Cinema	Sat 21 Mar, 7:30pm	Available

Included with Plus and above. Standard members pay £5.

Buttons: Save, Book.

Screen 3, web, account

Membership	Value
Tier	Plus
Price	£96 / year
Renews	14 Aug 2027

Household	Role
Anna Reid	Primary
Mark Boyd	Adult
Ella Reid	Child, age 9, no login

Booking history	Event	Venue
12 Mar 2027	Italian Film Weekend	Grand Cinema
04 Feb 2027	Late Opening	City Gallery

Your task

Design the database. Tables, columns, keys, relationships. Talk through it as you go.

We're more interested in the reasoning than the diagram, so where there's a choice, say what the alternatives were and why you picked one.

PART 2, CURVEBALLS

In order. Roughly six minutes each. Ask the question, then stay quiet.

1. One event, two venues

> Italian Film Weekend plays at two venues across three dates. How does that fit your model?

Good: the event and its occurrences are separate tables. Venue and start time sit on the occurrence, not the event.

Weak: venue and time are columns on the event, so the same event has to be duplicated. Fine at this stage. Watch how they take it.

Nudge if needed: where does capacity live? Per occurrence.

2. Saving different kinds of thing

> The Saved row holds events, venues and artists. Soon it'll hold articles too. Where does that go?

Good: one saves table with a type and an ID, and they mention the catch, that the database can't enforce the reference. A table per type is also fine if they say why.

Weak: one table with five nullable columns and nothing enforcing which is filled.

You're listening for whether they name a downside at all, not for a specific answer.

3. The price changes

> On 1 January, Plus goes from £96 to £110. Existing members keep paying £96 until they renew. What happens?

Good: the price is recorded on the membership when it's bought, or prices live in their own table with dates. Either is fine.

Weak: they update the price on the tier. Ask what a refund looks like six months later.

4. Who the membership belongs to

> Anna pays. Mark and nine-year-old Ella are covered by it. Ella has no login.

Good: a person and a login are different records, and the membership belongs to the household rather than to Anna's account.

Weak: the membership hangs off a user, so Ella needs a fake account. Don't announce it. Ask who the membership belongs to and let them find it.

If they're quick: Mark moves out in June and starts his own membership. His old bookings stay his.

PART 3, SPARES

Only if they finish early.

History has to stay true. In August a member looks at a booking from March. The event has been renamed and the venue has closed. What does the screen show? Look for the title and venue name copied onto the booking, and a clear sense of why transaction records shouldn't just point at current state.

Sold out. Friday is full and twelve people are waiting. Someone cancels. Ordering, and what happens if two seats free up at once.

Reporting. Bookings by tier, monthly, across two hundred partners. Does that query run, and what would they do if it didn't.

Deletion. A member asks to be erased, but their bookings sit in a partner's revenue reports. Cascade or anonymise.

PART 4, REFERENCE MODEL

A yardstick, not a target. Good answers will differ.

Places and programme. `organisations`. `venues` belonging to an organisation. `events` belonging to an organisation, holding the title and description. `sessions` belonging to an event and a venue, holding the start time and capacity. Event to venue is many-to-many through sessions.

People and membership. `people`, a human, who may or may not have a login. `users`, credentials belonging to a person, with the relationship optional so a child has no user. `households` as the billing unit. `household_memberships` joining a person to a household with a role and a date range, which handles Mark leaving. `memberships` belonging to a household and a tier, holding the price paid and the dates. `tiers` (Standard, Plus, Premium), and either `tier_prices` with validity ranges or the price captured on the membership.

Bookings. `bookings` belonging to a session and a person.

Saves. `saves` belonging to a person, with a subject type and ID.

That's around nine tables. If a candidate produces twenty, ask what each one earns.

PART 5, SCORING

Score 1 to 4 as in stage one.

Area	What you're reading
Cardinality	Do one-to-many and many-to-many land in the right places, and do they spot that sessions carry their own data rather than being plumbing
Polymorphic judgment	Can they use it and say what it costs
Temporal data	Does the price change destroy history

Area	What you're reading
Identity modelling	Person versus login. A surprising number of senior candidates conflate these.
Handling pressure	When a curveball breaks their model, do they adapt or defend a sunk design
Asking first	Do they ask anything before committing

Outcome	Signal
Strong hire	Split event from session unprompted, named the polymorphic trade-off without being asked, kept price history intact
Hire	Solid cardinality, adapts well, may need a nudge on two of the four
Discuss	Good instincts, thin on integrity. Worth a second conversation on the gaps.
No	Model doesn't survive curveball 1 or 2, or they defend it rather than adjust

Weight cardinality, temporal data and identity modelling highest. That's the day-to-day work, in a codebase they didn't write, and getting those right is most of what keeps a schema habitable a year in.

But don't let a clean diagram outweigh handling pressure and asking first. Someone who gets to a good model by asking questions will keep getting to good models on problems this exercise doesn't cover. Someone who produces the same diagram from memory and defends it when curveball three breaks it won't.